

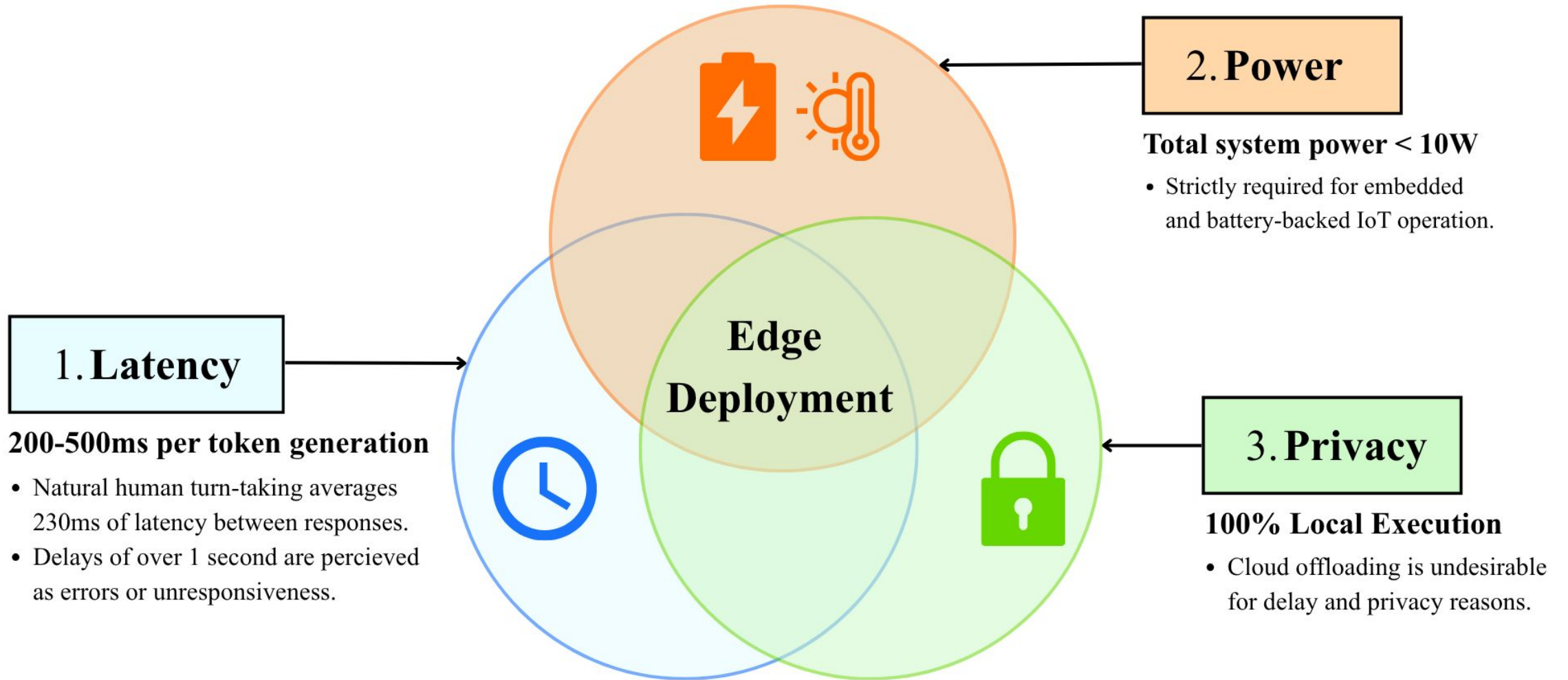


Tiny Acceleration of Liquid Foundation Models

Improving Model Inference Efficiency through Selective FPGA Offloading

Jacopo Marchesin

Why Edge AI for Conversational Audio?



Key Takeaway: conversational audio inference suits the edge, but obtaining competitive performance remains a challenge

Speech Model Profiling (GGUF)

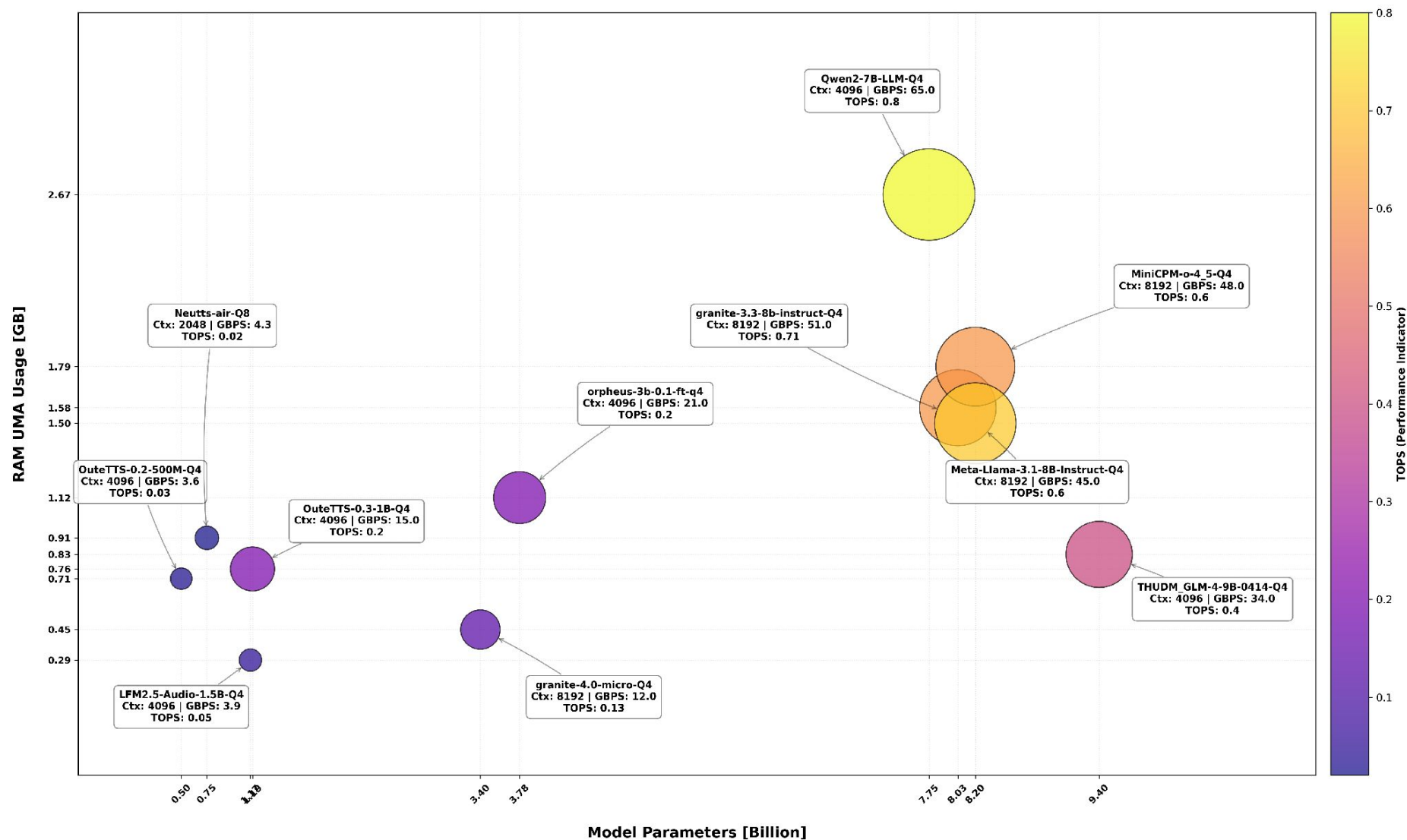


Fig. 1. Speech-to-speech AI benchmark @ 200 ms latency

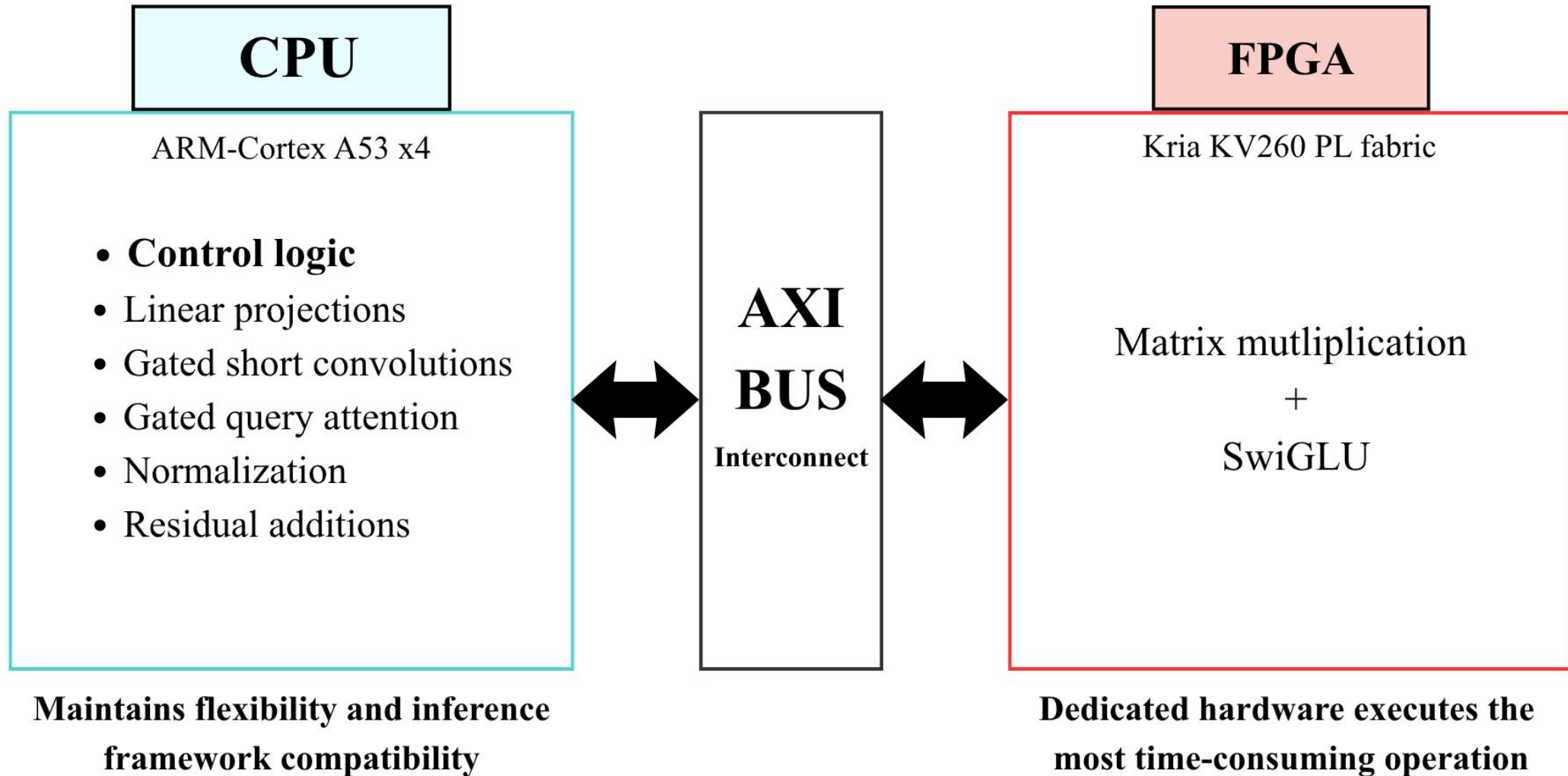
Parameter count, throughput and memory requirements need to be taken into account when targeting low-latency token generation.

CPU Execution Profile of the LFM2.5-1.2B language backbone

Architectural Block	Time (μs)	% Total
MatMul + SwiGLU	10,136,836	68.1%
Final Linear Projection	1,631,552	11.0%
Gated Short Convolution	1,555,396	10.4%
GQA Attention (MatMul)	1,318,155	8.9%
Other (norm, residual, RoPE...)	247,049	1.6%

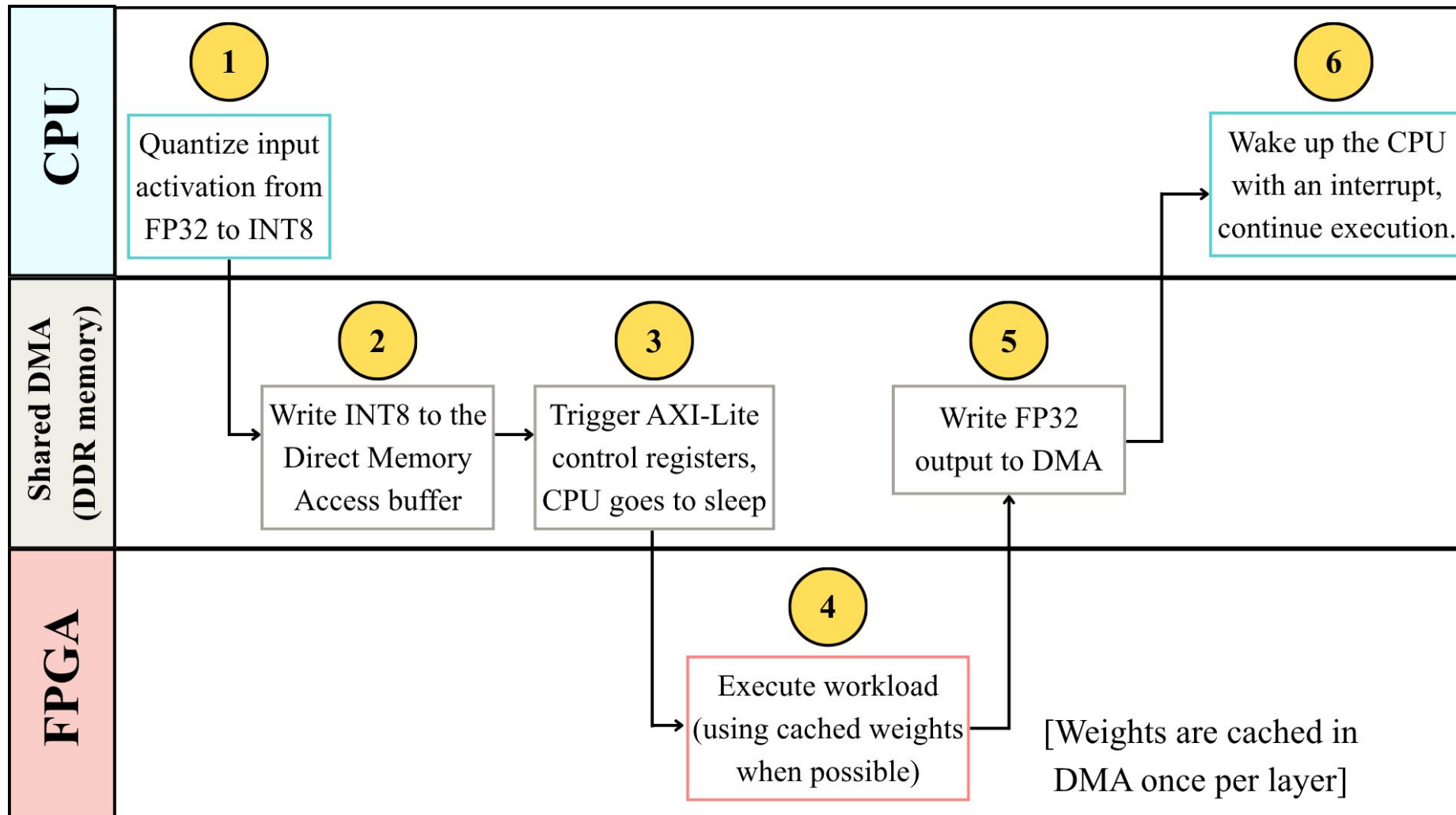
The MatMul + SwiGLU feedforward block is the most impactful target for selective FPGA acceleration.

Proposed system: FPGA offload



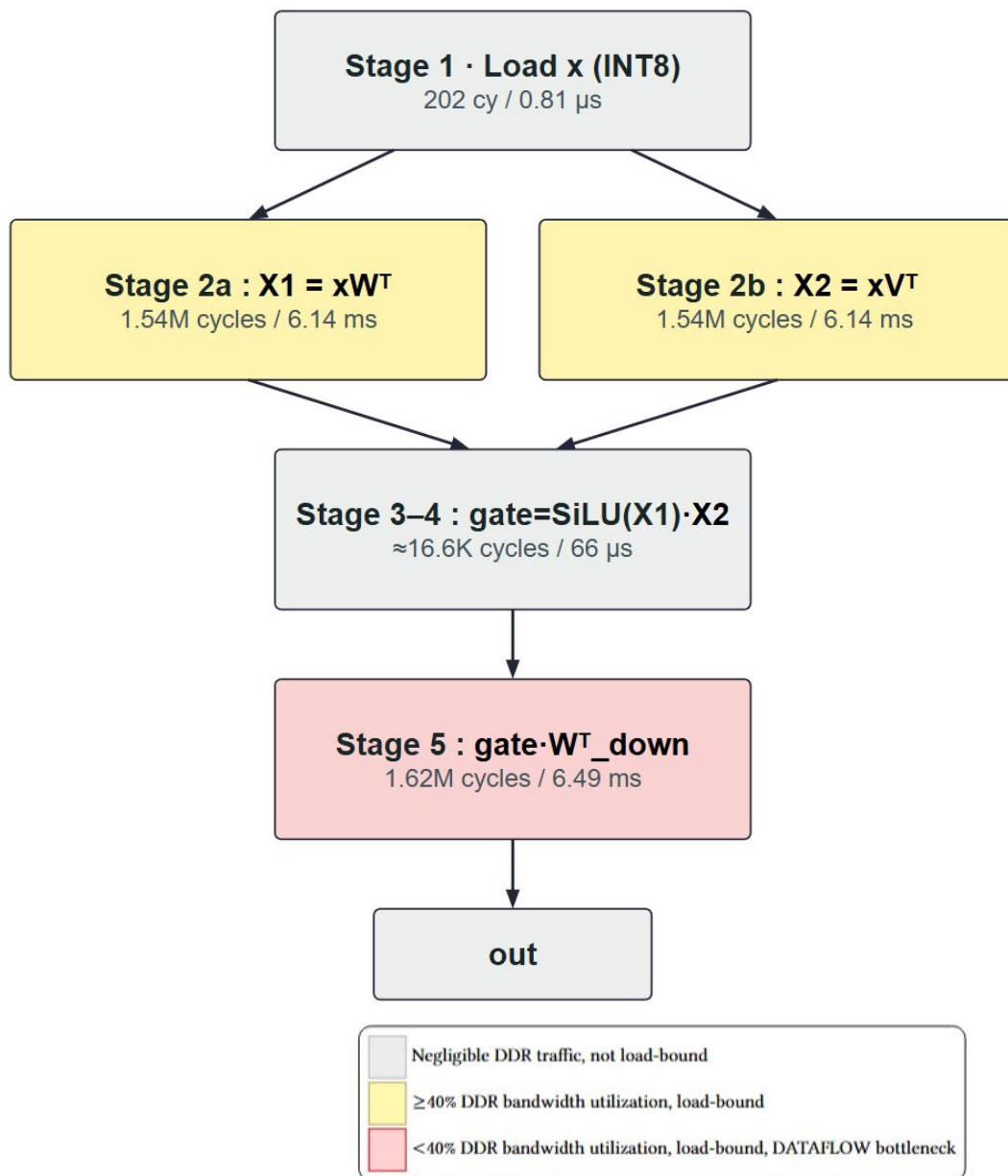
The operations not targeted for FPGA acceleration are executed by the CPU

Orchestration of the computational offload



- Weights cached once per layer in DMA-coherent buffer → never re-transferred for the same token
- CPU programs AXI-Lite registers, starts FPGA, sleeps until interrupt
- Fallback: LLAMA_SWIHW=0 → standard CPU path (full compatibility)

MatMul + SwiGLU Workload



$$\text{FFN}(x) = (\text{SiLU}(xW^T) \odot (xV^T)) W^T_{down}$$

- Five-stage HLS dataflow pipeline targeting interval = 1
 - Q4_K quantization: 6-bit sub-group scales, 16 $\times$ finer than Q4_0
 - Mixed precision: INT8 activations $\times$ 4-bit weights $\rightarrow$ 12-bit products
 - 32 parallel INT32 MAC chains (8 blocks $\times$ 4 rows) $\rightarrow$ 8.00 GOps/s pure compute throughput, of which 2.58 GOps/s is achieved.
 - CPU pre-decodes Q4_K headers $\rightarrow$ FPGA accesses nibbles via compile-time `.range()`, which eliminates LUT multiplexer trees.
-
- **Latency set by memory bandwidth, not arithmetic**
 - Stages 2a/2b: 41% of peak DDR BW (1.63 GB/s)
 - **Stage 5: 38% DDR BW $\rightarrow$ DATAFLOW bottleneck**
 - Total call interval: ~ 14 ms $\rightarrow$ ~ 224 ms per token

Throughput & Efficiency: FPGA+CPU vs CPU Only

T	Mode	Prefill t/s	Decode t/s	Power W	Prefill t/s/W	Decode t/s/W
T1	CPU Only	1.09	0.88	3.96	0.275	0.222
T1	FPGA+CPU	2.20	1.63	5.10	0.431	0.320
T2	CPU Only	2.17	1.66	4.25	0.511	0.391
T2	FPGA+CPU	2.90	2.21	5.15	0.563	0.429
T3	CPU Only	3.16	2.36	4.41	0.717	0.535
T3	FPGA+CPU	3.06	2.05	5.14	0.595	0.399
T4	CPU Only	3.52	2.31	4.28	0.822	0.540
T4	FPGA+CPU	2.85	1.46	5.13	0.556	0.285

Bold (blue) = better t/s/W within each thread-count pair

T1 efficiency with the FPGA accelerator : +57% prefill, +44% decode.

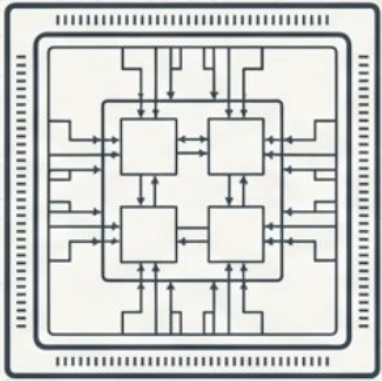
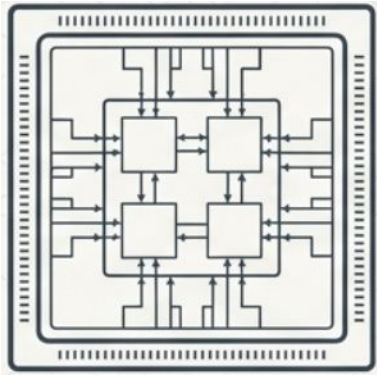
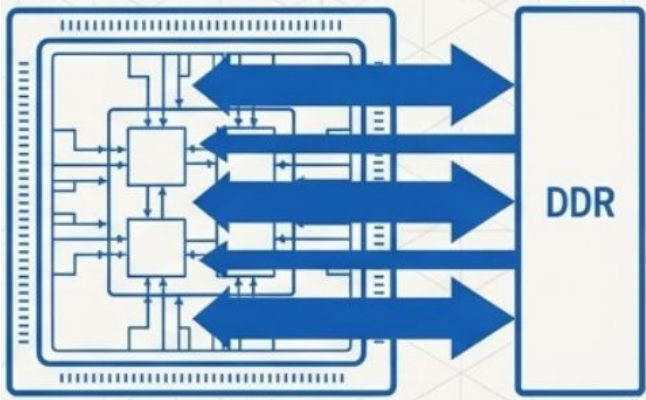
Real-Time Latency Compliance

Target: 200–500 ms per-token decode latency

Config	Decode Latency	< 500 ms?	Notes
T1 CPU Only	1136 ms	No	Baseline
T1 FPGA+CPU	613 ms	No	+44% eff.
T2 FPGA+CPU	452 ms	✓ Yes	2 threads only
T3 CPU Only	424 ms	✓ Yes	Needs 3 cores
T3 FPGA+CPU	488 ms	✓ Yes	–25% eff.
T4 CPU Only	433 ms	✓ Yes	Needs 4 cores

FPGA+CPU reaches sub-500 ms with only 2 threads, freeing cores for ASR & TTS

Comparison with other work on the same platform

Tiny Transformer	Tiled Attention	This Work
6.6k parameter ECG detection model	66M parameter DistilBERT model	LFM2.5-1.2B-Thinking model
Compute-bound	Compute-bound	Memory-bound
		
The model fits entirely on-chip because of its small size.	The model matrices fit entirely on-chip thanks to a two-level tiling strategy that uses BRAM effectively.	The model requires ~25MB in weights per layer, exceeding the ~7MB on-chip capacity, forcing reliance on DDR memory.

Prior art paradigms cannot be applied directly, as the model does not fit in the on-chip memory

Conclusions

- Selective FPGA offload of MatMul+SwiGLU is a viable step toward conversational edge inference
- Where FPGA path doesn't beat CPU-only, the results clarify why: performance is jointly governed by compute parallelism, data movement, and host-device orchestration
- Design already operates near the logic and routing budget of the KV260
- Further gains: memory-path restructuring, data movement and host-device orchestration.

Future work

- **Memory-centric optimization**

Pipelining, double-buffering, layout restructuring for higher sustained DDR bandwidth

- **Device migration**

ZU7EV: ~230K LUTs → fully unroll output MAC array, merge AXI load calls

- **Full pipeline integration**

Co-deploy ASR + LM + TTS within the same SoC power envelope

This work was presented as a poster at the 2026 annual meeting of the Italian Society of Electronics

